"Arquitectura Inicial - CLE"

Vega Ángeles Christopher

22140787

Moya Arreola Cristian Moya

22140757

Ruiz Avilés Fernanda Michelle

22140756

Casas Vázquez Luis Américo

22140750

Ingeniería en Sistemas Computacionales, Instituto Tecnológico de Querétaro.

Taller de Desarrollo Ágil de Software

José Miguel Jaime Ramírez

Índice

Visión General del Sistema 3
API Gateway, Red y Exposición 3
Microservicios Iniciales y Responsabilidades 3
usuarios-service 3
academico-service 4
infraestructura-service 4
inscripciones-service 4
Capa de Bases de Datos 4
Comunicación y Flujo de Integridad 5
Bibliografía 6

Visión General del Sistema

La plataforma CLE está construida bajo una arquitectura de microservicios orientada al dominio, diseñada para garantizar alta concurrencia, escalabilidad y tolerancia a fallos. El sistema se divide en dos bloques principales:

Frontend (Cliente): Una *Single Page Application* (SPA) reactiva desarrollada con React, Vite y Tailwind CSS, encargada de presentar las interfaces específicas para los tres roles del sistema (Administrador, Maestro y Alumno).

Backend (Servidor): Un conjunto de cuatro microservicios independientes desarrollados en Node.js con el framework Express, contenerizados con Docker y orquestados mediante Docker Compose para aislar y ejecutar la lógica de negocio.

API Gateway, Red y Exposición

La comunicación entre los clientes externos y la red interna de microservicios está controlada por capas de enrutamiento y seguridad:

Cloudflare Tunnel y Proxy Vite: Gestionan la exposición segura de la plataforma hacia internet y el enrutamiento inicial.

Nginx (API Gateway): Opera como el proxy inverso principal dentro de la instancia de Amazon EC2. Recibe las peticiones HTTP provenientes del frontend y, según las reglas de enrutamiento de URL, las dirige al contenedor Docker del microservicio correspondiente, ocultando la topología interna de la red.

Microservicios Iniciales y Responsabilidades

Para mantener bajo acoplamiento y responsabilidades claras, el backend opera con cuatro microservicios independientes:

usuarios-service

Responsabilidades: Controla el acceso al sistema mediante un esquema RBAC (*Role-Based Access Control*). Gestiona el inicio de sesión, creación de cuentas, cifrado de contraseñas (hash) y la emisión/validación de tokens JWT (Bearer Auth).

Base de datos asignada: cle_usuarios (3 tablas).

academico-service

Responsabilidades: Centraliza la lógica educativa de la coordinación. Administra el catálogo de idiomas, niveles, unidades de evaluación, expedientes de alumnos (internos del ITQ y externos) y el historial de certificaciones de los docentes.

Base de datos asignada: cle_academico (9 tablas).

infraestructura-service

Responsabilidades: Administra la disponibilidad de los recursos físicos del ITQ destinados al CLE (campus, aulas y laboratorios), controlando capacidades y estatus operativo.

Base de datos asignada: cle_infraestructura (2 tablas).

inscripciones-service

Responsabilidades: Gestiona la operación académica por ciclo escolar. Controla la apertura de periodos, formación de grupos, asignación de horarios sin empalmes, monitoreo de cupos críticos en tiempo real y captura de calificaciones por unidad.

Base de datos asignada: cle_inscripciones (5 tablas).

Capa de Bases de Datos

Se implementa el patrón arquitectónico *Database-per-Service* para asegurar la independencia de datos entre microservicios:

Infraestructura Cloud: La persistencia recae en el motor relacional PostgreSQL, alojado en una única instancia administrada de Amazon RDS (cle-postgresql).

Segmentación Lógica: Dentro de esta instancia coexisten 4 bases de datos lógicas independientes, sumando un total de 19 tablas en el alcance inicial.

Comunicación y Flujo de Integridad

Comunicación Externa (Cliente-API): El frontend consume los servicios de manera asíncrona mediante peticiones HTTP RESTful utilizando Axios, adjuntando el token JWT en las cabeceras a través de interceptores de sesión.

Flujo Interno por Capas: Dentro de cada microservicio, las peticiones siguen un flujo estricto:Routes (Validación Zod + Middlewares JWT) → Controllers → Services → Repositories → PostgreSQL

Integridad Referencial Cruzada: Al estar separadas las cuatro bases de datos, no existen llaves foráneas físicas entre ellas. La vinculación entre entidades de distintos dominios (ej. asignar un maestro_id de cle_academico a un grupo en cle_inscripciones) se resuelve mediante referencias lógicas, cuya existencia e integridad se validan directamente en la capa de *Services* antes de confirmar cada transacción.

Bibliografía

Amazon Web Services. (2026). *Documentación oficial de AWS (Amazon EC2 y Amazon RDS)*. Recuperado de: https://docs.aws.amazon.com/es_es/

Docker Inc. (2026). *Docker Documentation: Guías y manuales de referencia*. Recuperado de: https://docs.docker.com/

JSON Web Token. (2026). *Introduction to JSON Web Tokens*. Recuperado de: https://jwt.io/introduction

PostgreSQL Global Development Group. (2026). *Manual oficial de PostgreSQL*. Recuperado de: https://www.postgresql.org/docs/